

Mock Interview: Senior System Analyst

Раунд 1 — Теоретическое интервью

КЛЮЧ С ОТВЕТАМИ (для интервьюера)

Длительность:	~120 минут (~100 вопросов)
Формат:	Вопрос, затем точный ответ. Уровни: Warm-up / Junior / Middle / Senior / Probe.
Раунды:	Раунд 1 (этот документ): только теория. Раунд 2 (отдельно): практика — живой SQL, разбор диаграмм, кейсы.

Шкала оценки

- 1 — Не знает / неверно
- 2 — Слышал термин, поверхностно
- 3 — Знает определение, но без глубины
- 4 — Уверенно, с примерами из практики
- 5 — Экспертно: объясняет trade-offs, крайние случаи, альтернативы

Как вести интервью

Для каждой темы иди от простого к сложному. Junior-вопрос — это разминка и проверка базы. Middle — проверка рабочего применения. Senior — проверка глубины: слушай не определение, а рассуждение о компромиссах, крайних случаях и реальном опыте. Если кандидат уверенно закрывает Senior-вопрос, можно пропустить более простые в этой теме и двигаться дальше.

Содержание

1. Разминка и калибровка (3)
2. Роль SA, основы профессии (6)
3. SDLC и методологии (8)
4. Стейкхолдеры и стратегический анализ (8)
5. Инженерия требований (12)
6. Моделирование процессов (BPMN) и UML (10)
7. Моделирование данных и SQL (теория) (12)
8. API и интеграции (10)
9. Системная архитектура и масштабирование (8)
10. NFR, безопасность, соответствие (8)
11. Тестирование, метрики, мониторинг, DevOps (awareness) (7)
12. Документация и инструменты (4)
13. Проверка реального опыта (не 'да/нет') (5)
14. AI и инструменты аналитика (7)

Всего вопросов: 108

1. Разминка и калибровка

Цель: понять контекст кандидата, снять напряжение, откалибровать уровень до глубоких вопросов. Не оценивай строго, слушай широту.

WARM-UP

Q1. Расскажи о себе за 2 минуты: где работаешь сейчас, какой продукт/домен, какая команда вокруг тебя (Dev, QA, PO, DevOps).

ОТВЕТ

Оцениваем не факты, а структуру ответа: называет ли домен, масштаб системы, свою зону ответственности, с кем взаимодействует. Senior обычно сразу даёт контекст бизнеса и своей роли в нём, а не перечисляет обязанности из вакансии.

WARM-UP

Q2. За что именно отвечаешь ты, а не команда, от начала задачи до релиза? Где начинается и заканчивается твоя ответственность?

ОТВЕТ

Senior владеет полным циклом: от прояснения проблемы и сбора требований до приёмки (UAT) и оценки результата. Красный флаг: ответ только 'пишу User Story по задаче от PO'. Хороший ответ показывает проактивность и владение end-to-end.

WARM-UP

Q3. Назови самый сложный проект за карьеру и в чём была именно аналитическая сложность (не техническая).

ОТВЕТ

Ищем: противоречивые стейкхолдеры, неясные требования, интеграция legacy-систем, миграция данных, регуляторика. Senior отделяет аналитическую сложность (неопределённость, конфликт интересов, домен) от инженерной. Слабый кандидат сведёт всё к 'было много задач'.

2. Роль SA, основы профессии

База профессии. Быстро проходим Junior, задерживаемся на границах ролей и ценности SA.

JUNIOR

Q4. Что делает системный аналитик одним предложением? Какие его основные артефакты (deliverables)?

ОТВЕТ

SA — мост между бизнесом и разработкой: превращает бизнес-потребности в понятные и проверяемые требования для команды. Артефакты: SRS/T3, User Story с Acceptance Criteria, BPMN/UML-диаграммы, ERD, спецификации API, RTM.

JUNIOR**Q5. Чем SA отличается от BA и от Product Manager?****ОТВЕТ**

BA работает ближе к бизнесу и процессам (зачем и что для бизнеса). SA ближе к системе (как это ляжет на архитектуру, данные, интеграции). Product Manager отвечает за продукт и его ценность/приоритеты (что и почему для рынка), владеет продуктовым видением и метриками. Границы размыты и зависят от компании.

MIDDLE**Q6. Чем отличается роль SA в продуктовой компании, в аутсорсе и во внутреннем IT (in-house)? Где ты работал?****ОТВЕТ**

Продукт: долгий горизонт, работа с метриками, гипотезами, один домен вглубь. Аутсорс: много доменов, жёсткие ТЗ и границы скоупа, формализация ради контракта и оплаты. In-house: знание внутренних процессов, интеграция с зоопарком legacy-систем, близко к конечным пользователям-сотрудникам. Senior осознаёт, как тип компании меняет его работу.

MIDDLE**Q7. Чем SA отличается от разработчика в понимании системы? Где проходит граница 'что' и 'как'?****ОТВЕТ**

SA отвечает за 'что система должна делать' и 'почему', разработчик за 'как это реализовать'. Но Senior SA понимает 'как' достаточно, чтобы писать реалистичные требования, оценивать трудоёмкость и не ставить невыполнимых NFR. Граница не жёсткая: SA не проектирует классы, но обязан понимать архитектурные ограничения.

SENIOR**Q8. Как ты измеряешь, что твоя работа как аналитика приносит ценность? Не 'написал ТЗ', а бизнес-эффект.****ОТВЕТ**

Senior связывает свою работу с исходами: меньше переделок (rework rate), меньше дефектов из-за неполных требований, быстрее time-to-market, точнее приёмка. Хороший ответ: 'плохое требование стоит в 10-100 раз дороже, если поймано на проде, а не на этапе анализа'. Оперировать cost of bad requirements.

SENIOR**Q9. Опиши ситуацию, где ты изменил решение команды или бизнеса своим анализом. Что было бы, не вмешайся ты?****ОТВЕТ**

Проверяем влияние и субъектность. Senior приводит конкретику: нашёл противоречие в требованиях, выявил невозможность интеграции, предложил дешевле/проще решение, остановил лишний скоуп. Слабый ответ — общие слова без последствий.

3. SDLC и методологии

Проверяем понимание жизненного цикла и роли SA в разных методологиях. Junior — определения, Senior — выбор и компромиссы.

JUNIOR

Q10. Назови стадии SDLC и что происходит на каждой.

ОТВЕТ

Concept/Initiation, Requirements, Design, Development, Testing, Deployment, Maintenance, плюс Post-Implementation Review (оценка достигли ли обещанного). Junior должен назвать хотя бы основные и понимать, что это последовательность от идеи до сопровождения.

JUNIOR

Q11. Что такое client-server модель и почему она важна для аналитика?

ОТВЕТ

Клиент (frontend/приложение) шлёт запрос, сервер (backend) обрабатывает и отвечает. Это базовая структура почти любой системы: понимая её, SA правильно распределяет требования между слоями и понимает, где живут данные и логика.

MIDDLE

Q12. Чем Waterfall отличается от Agile? В каком проекте ты бы выбрал Waterfall осознанно?

ОТВЕТ

Waterfall: последовательные фазы, полное ТЗ заранее, изменения дорогие. Agile: итерации, требования уточняются по ходу, готовность к изменениям. Waterfall оправдан при фиксированном скоупе и контракте, жёсткой регуляторике, госзаказе, аппаратной зависимости. Senior не считает Waterfall 'плохим', а инструментом под контекст.

MIDDLE

Q13. Роли, церемонии и артефакты Scrum. В каких церемониях участвует SA и зачем?

ОТВЕТ

Роли: PO, Scrum Master, Dev Team. Церемонии: Sprint Planning, Daily, Review, Retro, Backlog Refinement. Артефакты: Product Backlog, Sprint Backlog, Increment. SA активно участвует в Refinement (готовит и уточняет требования) и Planning; на Review показывает соответствие требованиям.

MIDDLE

Q14. Kanban против Scrum: в чём разница и когда Kanban уместнее? Что такое WIP-лимит?

ОТВЕТ

Scrum — итерации фиксированной длины с обязательствами на спринт. Kanban — непрерывный поток без спринтов, вытягивание задач. Kanban уместен для поддержки, потока входящих запросов, команд с непредсказуемым приоритетом. WIP-лимит ограничивает число задач в работе одновременно, чтобы вскрыть узкие места и не распылять фокус.

SENIOR

Q15. Что такое Story Points и почему оценивают в них, а не в часах? Что такое Velocity и как её НЕ надо использовать?

ОТВЕТ

Story Points — относительная оценка сложности/объёма/риска, а не времени; убирают иллюзию точности часов и учитывают неопределённость. Velocity — сколько SP команда закрывает за спринт, для прогноза. Нельзя: сравнивать velocity разных команд, использовать как KPI/давление (это ведёт к раздуванию оценок).

SENIOR

Q16. Чем Definition of Done отличается от Acceptance Criteria? Приведи пример, где путаница между ними приводит к проблеме.

ОТВЕТ

AC — условия приёмки конкретной истории (что именно должно работать). DoD — общий чек-лист готовности для любой задачи (код-ревью, тесты, документация, деплой на stage). Путаница: история 'работает по AC', но без тестов и документации по DoD — формально принята, реально не готова к релизу. Это разные уровни.

SENIOR

Q17. Что такое RAID-лог и HADI-цикл? Какова роль SA в каждом?

ОТВЕТ

RAID: Risks, Assumptions, Issues, Dependencies — журнал управления неопределённостью; SA вносит вклад (особенно assumptions и dependencies), но не владеет им единолично (это чаще PM). HADI: Hypothesis-Action-Data-Insights — цикл продуктовой проверки гипотез; SA помогает формулировать проверяемую гипотезу и метрику успеха, а также фиксировать инсайты.

4. Стейкхолдеры и стратегический анализ

Работа с людьми и обоснование 'зачем' до 'что'. Senior обязан уметь обосновывать проект деньгами.

JUNIOR

Q18. Кто такие стейкхолдеры? Приведи примеры для любого продукта.

ОТВЕТ

Все, кто влияет на проект или на кого влияет проект: заказчик, конечные пользователи, бизнес-владелец, команда разработки, регулятор, поддержка, подрядчики. Junior должен различать хотя бы Client / User / Business.

JUNIOR

Q19. Что такое RACI-матрица и какие роли она описывает?

ОТВЕТ

Responsible (исполняет), Accountable (отвечает, один на задачу), Consulted (консультируют, двусторонняя связь), Informed (уведомляют). Распределяет ответственность по задачам, чтобы не было 'все и никто'.

MIDDLE

Q20. Что такое Power-Interest матрица и как ты действуешь с каждым квадрантом?

ОТВЕТ

Оси: власть (влияние) и интерес. High power/high interest — управлять плотно (manage closely). High power/low interest — держать удовлетворёнными. Low power/high interest — держать информированными. Low power/low interest — минимальный мониторинг. Определяет стратегию коммуникации, а не отношение к людям.

MIDDLE

Q21. AS-IS и TO-BE: что это и зачем описывать текущее состояние, если мы всё равно строим новое?

ОТВЕТ

AS-IS — как процесс работает сейчас, TO-BE — как должен работать. Без AS-IS не увидишь реальную боль, скрытые кейсы, интеграции и не сможешь оценить масштаб изменений и обучение людей. TO-BE без AS-IS часто игнорирует существующие ограничения.

MIDDLE

Q22. Что такое Gap Analysis и как он связывает AS-IS и TO-BE с требованиями?

ОТВЕТ

Gap Analysis — разница между текущими и желаемыми возможностями. Каждый разрыв (gap) превращается в требование или инициативу. Это мост: AS-IS + TO-BE дают разрывы, разрывы дают скоуп работ.

SENIOR

Q23. Чем отличается Business Case от Business Model Canvas? Что SA вносит в Business Case?

ОТВЕТ

BMC — снимок бизнес-модели на 9 блоков (кто клиенты, ценность, каналы, потоки доходов и т.д.), помогает понять контекст. Business Case — обоснование конкретной инвестиции: проблема, варианты, затраты, выгоды, риски, рекомендация. SA вносит скоуп, оценку трудоёмкости, риски, операционные эффекты, но не финансовое решение целиком.

SENIOR

Q24. Объясни Cost-Benefit Analysis: как ты структурируешь затраты и выгоды? Что такое TCO?

ОТВЕТ

СБА сопоставляет затраты и выгоды. Затраты: единоразовые (разработка, лицензии, миграция) против регулярных (поддержка, инфраструктура). Выгоды: осязаемые (экономия, рост выручки) против неосязаемых (удобство, бренд). TCO (Total Cost of Ownership) — полная стоимость владения за жизненный цикл, обычно 3 года, включая скрытые операционные расходы, а не только цену внедрения.

SENIOR

Q25. Feasibility Analysis: какие виды осуществимости ты проверяешь и приведи пример, где проект технически возможен, но не осуществим по другому измерению.

ОТВЕТ

Операционная (примут ли люди/процессы), техническая (можем ли построить), финансовая/экономическая (окупится ли). Пример: технически можно внедрить новую CRM, но операционно провал — сотрудники саботируют, нет обучения; или финансово TCO выше выгоды. Senior понимает, что 'можем построить' не равно 'стоит строить'.

5. Инженерия требований

Ядро профессии. Самая большая секция. Junior — типы и качество, Senior — управление изменениями, приоритизация, трассируемость.

JUNIOR

Q26. Что такое требование? Назови типы: Business, User, Functional, Non-Functional с примером каждого.

ОТВЕТ

Требование — задокументированная потребность к системе. Business: 'снизить время оформления заказа на 30%'. User: 'как покупатель, хочу отслеживать статус заказа'. Functional: 'система отправляет email при смене статуса'. Non-Functional: 'страница грузится ≤ 2с при 95-м перцентиле'.

JUNIOR

Q27. Что означает SMART/качественное требование? Перепиши плохое требование 'система должна быть быстрой'.

ОТВЕТ

Требование должно быть конкретным, измеримым, достижимым, релевантным, проверяемым, недвусмысленным. Переписать: 'Время ответа API поиска ≤ 500 мс при 95-м перцентиле под нагрузкой 1000 RPS'. Ключ: измеримость и проверяемость.

JUNIOR

Q28. Чем FR отличается от NFR и почему в спеке нужны оба?

ОТВЕТ

FR — что система делает (функции, поведение). NFR — какова система при этом (скорость, безопасность, доступность, масштабируемость, удобство, соответствие). Без NFR можно построить функционально верную, но непригодную систему (медленную, небезопасную). NFR часто определяют архитектуру.

MIDDLE**Q29. Что делает User Story хорошей? Расшифруй INVEST.****ОТВЕТ**

INVEST: Independent (независима), Negotiable (обсуждаема), Valuable (ценна для пользователя), Estimable (оценима), Small (небольшая, влезает в спринт), Testable (проверяема). Хорошая история несёт ценность, а не техническую подзадачу, и имеет проверяемые AC.

MIDDLE**Q30. Формат Given-When-Then для Acceptance Criteria: напиши пример для 'вход по паролю'.****ОТВЕТ**

Given пользователь на странице входа с верным email; When вводит корректный пароль и нажимает 'Войти'; Then система пускает в личный кабинет. Плюс негативный сценарий: Given ... When вводит неверный пароль 5 раз Then аккаунт блокируется на 15 минут. Senior всегда добавляет негативные и граничные сценарии.

MIDDLE**Q31. Когда ты используешь Use Case спецификацию вместо User Story? В чём разница?****ОТВЕТ**

User Story — короткая, для Agile-беклога, ценность + AC. Use Case — детальный сценарий взаимодействия актора и системы с основным и альтернативными потоками, пред- и постусловиями. Use Case уместен для сложной логики с множеством ветвлений, в регулируемых/крупных проектах. Они дополняют, а не исключают друг друга.

MIDDLE**Q32. Техники сбора требований (elicitation): назови минимум четыре и когда какая лучше.****ОТВЕТ**

Интервью (глубина, 1-на-1), воркшопы (согласование многих стейкхолдеров разом, разрешение конфликтов), наблюдение (вскрывает реальный процесс против декларируемого), опросы (много респондентов, количественно), анализ документов (regs, старые ТЗ, тикеты). Senior подбирает технику под ситуацию, а не использует только интервью.

MIDDLE**Q33. Root Cause Analysis: что такое 5 Why и диаграмма Ишикавы (Fishbone)?****ОТВЕТ**

5 Why — последовательно спрашиваем 'почему' ~5 раз, докапываясь от симптома до корневой причины. Fishbone/Ишикава — визуальная группировка возможных причин по категориям (люди, процессы, технологии, данные и т.д.). Цель обеих — лечить причину, а не симптом; SA применяет до того, как писать решение.

SENIOR

Q34. Объясни MoSCoW и модель Капо. Чем они дополняют друг друга при приоритизации?

ОТВЕТ

MoSCoW: Must/Should/Could/Won't — приоритет для скоупа релиза. Капо классифицирует фичи по влиянию на удовлетворённость: базовые (must-be, их отсутствие бесит), линейные (performance, чем больше тем лучше), восхищающие (delighters, неожиданная радость). MoSCoW отвечает 'что делаем сейчас', Капо — 'как фича влияет на восприятие'. Вместе: не тратьте Must-бюджет на delighters, пока нет базовых.

SENIOR

Q35. Что такое Scope Creep и как формальный процесс Change Request его контролирует? Опиши шаги.

ОТВЕТ

Scope Creep — неконтролируемое расползание скоупа мелкими 'ну ещё чуть-чуть'. Контроль через CR-процесс: запрос изменения, оценка влияния (impact analysis) на сроки/бюджет/другие системы, решение (одобрить/отклонить) уполномоченным, обновление RTM и спеки. Senior не говорит 'нет' изменениям, а делает их видимыми и оценёнными.

SENIOR

Q36. Что такое Impact Analysis и Requirements Traceability Matrix? Как они работают вместе при изменении требования?

ОТВЕТ

Impact Analysis оценивает downstream-эффект изменения на другие требования, системы, команды. RTM — матрица трассировки: связывает бизнес-требование → функциональное → дизайн → тест-кейс. При CR смотришь в RTM, кого затрагивает изменение, оцениваешь impact по этим связям, обновляешь всю цепочку. Без RTM impact analysis превращается в угадывание.

SENIOR

Q37. Organizational Change Management: когда система выходит в прод, что меняется кроме кода, и в чём тут роль SA?

ОТВЕТ

Меняются роли, рабочие процессы, зоны ответственности людей. SA оценивает change impact (кого и как затронет), пишет план коммуникаций для go-live, определяет потребность в обучении (training needs analysis) и передаёт тренеру, но не владеет самим управлением изменениями целиком (это Change Manager). Senior понимает: внедрить систему технически недостаточно, если люди не перестроятся.

6. Моделирование процессов (BPMN) и UML

Проверяем нотации. Теория здесь; разбор реальных диаграмм пойдёт в практическом раунде.

JUNIOR

Q38. Что такое BPMN и зачем аналитику моделировать процесс до написания требований?

ОТВЕТ

BPMN — стандартная нотация моделирования бизнес-процессов. Моделируем до требований, чтобы увидеть, как бизнес работает сейчас, найти узкие места и договориться с стейкхолдерами об общей картине, прежде чем специфицировать систему. Диаграмма снимает недопонимание лучше текста.

JUNIOR

Q39. Назови базовые элементы BPMN: события, задачи, шлюзы.

ОТВЕТ

События (Start, Intermediate, End) — что происходит. Задачи (Task) — единица работы. Шлюзы (Gateway) — точки ветвления/слияния потока. Плюс потоки (Sequence Flow) и дорожки/пулы (Lanes/Pools).

MIDDLE

Q40. Разница между XOR, AND и OR шлюзами? Приведи пример для каждого.

ОТВЕТ

XOR (эксклюзивный) — идёт ровно одна ветка по условию (оплата картой ИЛИ наличными). AND (параллельный) — идут все ветки одновременно (одновременно резервируем товар И шлём подтверждение). OR (инклюзивный) — одна или несколько веток по условиям (уведомить email и/или SMS в зависимости от настроек). Senior помнит про слияние: параллельные ветки надо свести AND-шлюзом.

MIDDLE

Q41. Разница между Pool и Lane, между Sequence Flow и Message Flow?

ОТВЕТ

Pool — отдельный участник процесса (компания, система). Lane — роль/подразделение внутри пула. Sequence Flow (сплошная стрелка) — порядок шагов внутри одного пула. Message Flow (пунктир) — обмен сообщениями между пулами. Ошибка новичка: соединять пулы sequence flow.

MIDDLE

Q42. Когда использовать Use Case диаграмму, а когда Sequence? Что каждая показывает?

ОТВЕТ

Use Case — что система делает и кто (акторы) с ней взаимодействует, границы системы, на верхнем уровне. Sequence — как во времени объекты/участники обмениваются сообщениями для одного сценария, по шагам. Use Case отвечает 'что и кто', Sequence — 'как именно и в каком порядке'.

MIDDLE**Q43. В Use Case диаграмме: разница между include, extend и generalization?****ОТВЕТ**

Include — обязательное переиспользование общего поведения (оформление заказа всегда include 'проверить оплату'). Extend — опциональное расширение при условии ('применить промокод' расширяет оформление). Generalization — наследование актора или use case (обычный и VIP-пользователь). Частая путаница include/extend: include всегда, extend по условию.

SENIOR**Q44. На Sequence-диаграмме: синхронные и асинхронные сообщения, activation box, фрагменты alt/opt/loop. Зачем это SA?****ОТВЕТ**

Синхронное сообщение (закрашенная стрелка) — отправитель ждёт ответа; асинхронное (открытая стрелка) — не ждёт. Activation box — период активности участника. Фрагменты: alt (ветвление if/else), opt (опциональный блок), loop (цикл), ref (ссылка на другой сценарий). SA использует это, чтобы точно специфицировать интеграции, особенно порядок вызовов и что делать при таймауте.

SENIOR**Q45. Чем Class-диаграмма отличается от ERD? Что такое множественность (multiplicity)?****ОТВЕТ**

Class-диаграмма моделирует объекты с атрибутами И поведением (методами) и их связи, ближе к коду/домену. ERD моделирует структуру хранения данных: сущности, атрибуты, ключи, связи, без поведения. Multiplicity — сколько экземпляров одного класса связано с другим (1, 0..1, 1..*, *). Senior знает: одна доменная модель != одна модель хранения (impedance mismatch).

SENIOR**Q46. Когда State Machine диаграмма лучше Activity-диаграммы?****ОТВЕТ**

Activity показывает поток работ/процесс (последовательность действий, ветвления). State Machine показывает жизненный цикл одного объекта: его состояния и переходы по событиям (заказ: создан → оплачен → отправлен → доставлен → отменён). State Machine выбираем, когда важны состояния объекта и допустимые переходы (что запрещено), а не поток действий.

SENIOR**Q47. Чем swimlane в Activity-диаграмме отличается от Lane в BPMN? И когда ты вообще выберешь BPMN, а когда UML Activity?****ОТВЕТ**

Концептуально похожи (обе показывают, кто выполняет шаг), но BPMN — стандарт именно для бизнес-процессов, богаче на события/шлюзы/сообщения и понятнее бизнесу. UML Activity ближе к системной/программной логике и живёт в наборе UML. Senior: для бизнес-стейкхолдеров и межсистемного взаимодействия — BPMN; для внутренней логики фичи в контексте другой UML-документации — Activity.

7. Моделирование данных и SQL (теория)

SQL здесь только теоретически; живые запросы будут в практическом раунде. Проверяем понимание модели данных и поведения БД.

JUNIOR

Q48. Что такое ERD? Объясни сущность, атрибут, первичный и внешний ключ.

ОТВЕТ

ERD — диаграмма сущность-связь, модель структуры данных. Сущность — объект учёта (Клиент, Заказ). Атрибут — свойство (имя, дата). Primary Key — уникальный идентификатор записи. Foreign Key — ссылка на первичный ключ другой таблицы, создаёт связь.

JUNIOR

Q49. Что такое кардинальность связи? Объясни 1:1, 1:N, M:N.

ОТВЕТ

Кардинальность — сколько записей одной сущности связано с другой. 1:1 (у человека один паспорт), 1:N (у клиента много заказов), M:N (у студента много курсов, у курса много студентов). M:N в реляционной БД реализуется через промежуточную (bridge) таблицу.

JUNIOR

Q50. Разница между Natural и Surrogate первичным ключом? Что обычно предпочтительнее и почему?

ОТВЕТ

Natural — ключ из реальных данных (ИНН, email). Surrogate — искусственный (автоинкремент id, UUID), без бизнес-смысла. Обычно предпочитают surrogate: не меняется при смене бизнес-данных, стабилен, проще для связей. Natural-ключ рискован, если 'уникальное' значение вдруг меняется.

MIDDLE

Q51. Что такое нормализация и какие аномалии она предотвращает? Кратко 1NF, 2NF, 3NF.

ОТВЕТ

Нормализация устраняет избыточность, предотвращая аномалии вставки, обновления и удаления. 1NF — атомарные значения, без повторяющихся групп. 2NF — 1NF плюс нет частичной зависимости от части составного ключа. 3NF — 2NF плюс нет транзитивных зависимостей между неключевыми полями. Senior называет именно три аномалии, а не только 'убираем дубли'.

MIDDLE

Q52. Что такое денормализация и когда она оправдана?

ОТВЕТ

Денормализация — намеренное введение избыточности ради скорости чтения (меньше JOIN-ов). Оправдана при read-heavy нагрузке, отчётах, аналитике, кэшах, где стоимость лишних JOIN выше стоимости дублирования. Плата: усложняется поддержание целостности при записи. Это сознательный trade-off write-integrity против read-performance.

MIDDLE

Q53. Разница между WHERE и HAVING? И между INNER и LEFT JOIN — что вернётся?

ОТВЕТ

WHERE фильтрует строки ДО группировки; HAVING фильтрует уже агрегированные группы ПОСЛЕ GROUP BY. INNER JOIN вернёт только совпавшие в обеих таблицах строки; LEFT JOIN вернёт все строки левой таблицы плюс совпадения справа, а где нет совпадения — NULL. Классика: 'найти клиентов без заказов' = LEFT JOIN + WHERE order.id IS NULL.

MIDDLE

Q54. Что такое ACID? Объясни каждое свойство простыми словами.

ОТВЕТ

Atomicity — транзакция целиком или никак (перевод денег: списание и зачисление вместе). Consistency — БД переходит из корректного состояния в корректное, правила не нарушаются. Isolation — параллельные транзакции не мешают друг другу. Durability — после commit данные сохранены даже при сбое питания.

SENIOR

Q55. Что такое CTE и оконные функции (window functions)? Зачем разработчику ROW_NUMBER или LAG, и что тут важно знать SA?

ОТВЕТ

CTE (WITH ...) — именованный временный результат для читаемости и рекурсии. Оконные функции считают по 'окну' строк без схлопывания в группы (в отличие от GROUP BY): ROW_NUMBER нумерует, RANK ранжирует, LAG/LEAD берут предыдущую/следующую строку. SA важно понимать, что такие вычисления (топ-N, running total, дедупликация) возможны в БД, чтобы грамотно формулировать требования к отчётам, а не заставлять делать это в коде.

SENIOR

Q56. Что такое индекс и как он ускоряет и ЗАМЕДЛЯЕТ работу БД? Когда индекс не поможет?

ОТВЕТ

Индекс — структура (обычно B-дерево), ускоряющая поиск/сортировку по колонке ценой места и замедления записи (каждый INSERT/UPDATE обновляет индексы). Не поможет: на маленьких таблицах, при низкой селективности (колонка 'пол'), при функциях над колонкой в WHERE, при LIKE '%x'. Senior понимает: индексы — trade-off чтения против записи, а не бесплатное ускорение.

SENIOR

Q57. Реляционная БД против NoSQL: как ты выбираешь? Приведи связь с NFR и объясни CAP-теорему.

ОТВЕТ

Реляционная — фиксированная схема, ACID, сложные связи и транзакции. NoSQL — гибкая схема, горизонтальное масштабирование, специализация (документ/ключ-значение/колоночная/граф). Выбор идёт от NFR: строгая согласованность и транзакции → SQL; огромный масштаб и гибкость → NoSQL. CAP: при сетевом разделении (Partition) распределённая система выбирает между Consistency и Availability; нельзя иметь всё три одновременно при разделении. Отсюда CP- и AP-системы.

SENIOR

Q58. Что такое eventual consistency и когда бизнес готов её принять? Приведи пример, где НЕЛЬЗЯ.

ОТВЕТ

Eventual consistency — реплики принимают записи и сходятся к единому состоянию позже (модель AP). Приемлемо: лайки, счётчики просмотров, лента соцсети, корзина. Нельзя: банковский баланс, списание со склада последнего товара, бронирование места — там нужна строгая согласованность, иначе двойная продажа. Senior привязывает выбор к бизнес-рisku.

SENIOR

Q59. Твоя роль в миграции БД: что ты специфицируешь? Разница ETL и ELT?

ОТВЕТ

SA пишет спецификацию маппинга данных (поле-источник → поле-приёмник, трансформации), правила валидации, критерии отката (rollback), критерии успеха миграции. ETL — Extract-Transform-Load: трансформируем ДО загрузки (когда целевое хранилище со строгой схемой). ELT — грузим сырьё, трансформируем ВНУТРИ хранилища (типично для data warehouse/озёр с большой вычислительной мощностью). Senior знает, что миграция без rollback-плана и валидации — это риск.

8. API и интеграции

Как системы разговаривают. Ключевая зона для Senior SA, работающего с интеграциями.

JUNIOR

Q60. Что такое API простыми словами? Приведи аналогию.

ОТВЕТ

API — контракт, через который одна программа обращается к другой, не зная её внутренностей. Аналогия: официант в ресторане — ты (клиент) не идёшь на кухню (сервер), а делаешь заказ через официанта (API) по меню (документация). Важно: API скрывает реализацию за интерфейсом.

JUNIOR

Q61. Назови основные HTTP-методы и что каждый делает.

ОТВЕТ

GET — получить данные (без изменений). POST — создать новый ресурс. PUT — заменить/обновить целиком. PATCH — частично обновить. DELETE — удалить. Junior должен связать методы с CRUD (Create-POST, Read-GET, Update-PUT/PATCH, Delete-DELETE).

JUNIOR

Q62. Что означают группы HTTP-статусов 2xx, 4xx, 5xx? Приведи по одному коду.

ОТВЕТ

2xx — успех (200 OK, 201 Created). 3xx — перенаправление (301). 4xx — ошибка клиента (400 Bad Request, 401 Unauthorized, 404 Not Found). 5xx — ошибка сервера (500 Internal Server Error, 503 Service Unavailable). Ключ: 4xx — виноват запрос, 5xx — виноват сервер.

MIDDLE

Q63. Что такое REST и его основные принципы? Что значит 'stateless'?

ОТВЕТ

REST — архитектурный стиль поверх HTTP: ресурсы с URI, стандартные методы, представления (JSON). Принципы: клиент-сервер, единообразный интерфейс, кэшируемость, многоуровневость и главное stateless. Stateless: сервер не хранит состояние сессии между запросами, каждый запрос самодостаточен (несёт токен/контекст). Это упрощает масштабирование.

MIDDLE

Q64. Что такое идемпотентность? Какие HTTP-методы идемпотентны и почему это важно для интеграций?

ОТВЕТ

Идемпотентность — повторный одинаковый запрос даёт тот же результат, что и один. GET, PUT, DELETE идемпотентны; POST — нет (два POST создадут два заказа). Важно при сбоях сети: если ответ потерялся, безопасно повторить идемпотентный запрос. Для неидемпотентных нужен idempotency key. Senior поднимает эту тему сам при обсуждении платежей.

MIDDLE

Q65. Синхронное и асинхронное взаимодействие: разница и когда выбрать асинхронное?

ОТВЕТ

Синхронно — клиент шлёт запрос и ждёт ответа сразу (REST-вызов). Асинхронно — клиент отправляет и не ждёт, результат придёт позже (через очередь/событие/webhook). Асинхронное выбираем для долгих операций (генерация отчёта), высокой нагрузки, слабой связанности систем, устойчивости к пикам. Плата: сложнее отслеживать статус и ошибки.

SENIOR

Q66. REST против SOAP против gRPC против GraphQL: когда что? Что SA специфицирует по-разному?

ОТВЕТ

REST — универсален, человекочитаем, для большинства web API. SOAP — строгий контракт (WSDL), транзакции, legacy/банки/энтерпрайз. gRPC — бинарный, быстрый, для межсервисного общения (микросервисы), контракт в protobuf. GraphQL — клиент сам запрашивает нужные поля, гибко, решает over/under-fetching. SA по-разному документирует: для REST — эндпоинты/методы, для gRPC/SOAP — контракт-схему, для GraphQL — схему типов и запросы, а не фиксированные эндпоинты.

SENIOR

Q67. Три способа реального времени: polling, SSE, WebSocket. Как выбрать и как ты напишешь NFR?

ОТВЕТ

Polling — клиент периодически спрашивает (просто, но задержка и лишняя нагрузка). SSE — сервер шлёт поток обновлений в одну сторону (уведомления, лента). WebSocket — двусторонний постоянный канал (чат, торги, совместное редактирование). Выбор от частоты и направления обмена. NFR: вместо 'реалтайм' пишем 'обновление статуса доставляется клиенту ≤ 1 с, двусторонний канал, до N одновременных соединений'.

SENIOR

Q68. Как ты подходишь к версионированию API? Почему нельзя просто менять контракт?

ОТВЕТ

Нельзя ломать существующих потребителей: изменение контракта (убрать поле, сменить тип) сломает их интеграции. Версионирование (в URL /v2/, в заголовке, в media type) позволяет вводить breaking changes параллельно старой версии. Senior различает breaking и non-breaking изменения (добавить необязательное поле — не ломает) и планирует депрекацию старых версий.

SENIOR

Q69. Что такое message queue и event-driven архитектура? Какую проблему решает брокер сообщений? Kafka против RabbitMQ на уровне выбора.

ОТВЕТ

Очередь сообщений развязывает отправителя и получателя: producer кладёт сообщение, consumer забирает, когда готов. Решает пики нагрузки, устойчивость (сообщение не теряется), слабую связанность. Event-driven — компоненты реагируют на события, а не вызывают друг друга напрямую. RabbitMQ — классический брокер (умная маршрутизация, задачи, request/reply). Kafka — распределённый лог событий (высокий поток, хранение и переигрывание истории, аналитика/стриминг). Senior выбирает Kafka для потоков событий и переигрывания, RabbitMQ для маршрутизации задач.

9. Системная архитектура и масштабирование

Awareness-глубина для SA: не проектируем архитектуру, но понимаем её, чтобы писать реалистичные требования.

JUNIOR

Q70. Разница между frontend и backend? Что такое 3-уровневая архитектура?

ОТВЕТ

Frontend — то, что видит и с чем взаимодействует пользователь (интерфейс). Backend — серверная логика и обработка. 3-tier: презентационный слой (frontend) → слой логики (backend) → слой данных (БД). Разделение позволяет менять слои независимо.

JUNIOR**Q71. Что такое монолит и что такое микросервисы простыми словами?****ОТВЕТ**

Монолит — вся система одним большим приложением/кодовой базой, деплоится целиком. Микросервисы — система разбита на много мелких независимых сервисов, каждый со своей ответственностью и часто своей БД, деплоятся отдельно. Junior должен уловить 'один большой' против 'много маленьких'.

MIDDLE**Q72. Плюсы и минусы микросервисов против монолита? Когда монолит — правильный выбор?****ОТВЕТ**

Микросервисы: независимый деплой и масштабирование, изоляция отказов, разные технологии; но сложность распределённой системы, сеть, транзакции, эксплуатация. Монолит: проще разрабатывать, деплоить, отлаживать на старте; но тяжелеет с ростом. Монолит правилен для стартапа/MVP, малой команды, неясного домена. Senior: 'не начинай с микросервисов без причины' (monolith first).

MIDDLE**Q73. Горизонтальное против вертикального масштабирования? Что такое stateless-сервис и почему он важен для масштабирования?****ОТВЕТ**

Вертикальное — усилить одну машину (больше CPU/RAM), предел и дорого. Горизонтальное — добавить больше машин, почти безграничный, но нужен баланс нагрузки. Stateless-сервис не хранит состояние сессии у себя, поэтому любой инстанс обработает любой запрос, что позволяет свободно добавлять инстансы за балансировщиком. Stateful мешает горизонтальному масштабированию.

MIDDLE**Q74. Зачем нужны балансировщик нагрузки, кэш и CDN? Что каждый решает?****ОТВЕТ**

Load balancer распределяет запросы между инстансами (масштаб и отказоустойчивость). Кэш хранит частые данные ближе/быстрее (снижает нагрузку на БД, ускоряет ответ). CDN раздаёт статику (картинки, скрипты) с серверов близко к пользователю географически (снижает задержку). Все три — про производительность и масштаб.

SENIOR**Q75. Что такое SAGA-паттерн и какую проблему он решает в микросервисах? Что такое компенсирующая транзакция?****ОТВЕТ**

В микросервисах нет единой ACID-транзакции через несколько БД. SAGA — последовательность локальных транзакций, где при сбое шага запускаются компенсирующие транзакции, откатывающие уже сделанное (не rollback, а обратное действие: 'отменить бронь', 'вернуть деньги'). SA документирует шаги, компенсации и что делать при частичном сбое. Это про eventual consistency в бизнес-процессе.

SENIOR

Q76. Что такое C4-модель и зачем она аналитику?

ОТВЕТ

C4 — способ описывать архитектуру на 4 уровнях детализации: Context (система и её окружение), Container (приложения/сервисы/БД), Component (внутри контейнера), Code (классы). Даёт общий язык с архитекторами и позволяет показывать нужный уровень нужной аудитории (бизнесу — Context, разработчикам — Component). SA обычно живёт на уровнях Context и Container.

SENIOR

Q77. Как ты оцениваешь ожидаемую нагрузку для NFR? Покажи логику через DAU.

ОТВЕТ

Грубая формула: DAU (уникальных пользователей в день) × среднее число сессий × запросов за сессию, распределённое по пиковым часам, даёт RPS; плюс средний размер ответа даёт полосу. Пример: 100 000 DAU × 5 запросов, но 80% трафика в 4 пиковых часа → пиковый RPS считаем по пику, а не по среднему. Senior закладывает пик и запас, а не среднесуточное, и превращает это в конкретные NFR по throughput и latency.

10. NFR, безопасность, соответствие

Глубина по нефункциональным требованиям после архитектуры и API. Проверяем безопасность и комплаенс на уровне awareness Senior SA.

JUNIOR

Q78. В чём разница между аутентификацией и авторизацией?

ОТВЕТ

Аутентификация — кто ты (проверка личности: логин/пароль, токен). Авторизация — что тебе можно (права доступа к ресурсам после того, как личность подтверждена). Сначала authn, потом authz. Аналогия: authn — показать паспорт на входе, authz — какие двери твой ключ открывает.

JUNIOR

Q79. Что такое NFR по производительности? Разница между latency и throughput.

ОТВЕТ

Latency — время одного запроса от отправки до ответа (задержка). Throughput — сколько запросов система обрабатывает в единицу времени (пропускная способность, RPS). Можно иметь высокий throughput и высокую latency одновременно. NFR обязан задавать оба с числами и перцентилем.

MIDDLE

Q80. Что такое SLA, SLO (и SLI)? Как это связано с доступностью 99.9%?

ОТВЕТ

SLI — измеряемая метрика (например, доля успешных запросов). SLO — внутренняя цель по метрике (99.9% доступности). SLA — внешнее договорное обязательство перед клиентом с санкциями за нарушение. 99.9% ('три девятки') — это ~8.7 часов простоя в год; 99.99% — ~52 минуты. Senior переводит 'девятки' в реальные минуты простоя и понимает, что каждая девятка дорожает.

MIDDLE

Q81. Назови несколько пунктов OWASP Top 10 и как SA учитывает безопасность в требованиях, не будучи security-инженером.

ОТВЕТ

Например: Injection (SQL-инъекции), Broken Access Control, XSS, Broken Authentication. SA не пишет защиту, но формулирует security-NFR: валидация ввода, ролевая модель доступа, аудит-логи чувствительных действий, шифрование, ограничение попыток входа. Senior закладывает безопасность как требование, а не как 'разработчики сами сделают'.

MIDDLE

Q82. Что такое шифрование at rest и in transit? Что ты указываешь в требовании?

ОТВЕТ

In transit — данные шифруются при передаче по сети (TLS 1.2+/HTTPS). At rest — данные шифруются при хранении на диске/в БД (например AES-256). В требовании указываешь конкретику: 'весь трафик по TLS 1.2+', 'персональные данные хранятся зашифрованными AES-256', а не просто 'данные должны быть защищены'.

SENIOR

Q83. Объясни на уровне схемы OAuth 2.0 Authorization Code flow и что такое JWT. Где SA это применяет?

ОТВЕТ

OAuth 2.0 Authorization Code: пользователь логинится у провайдера, приложение получает временный code, обменивает его на access token (и refresh token) на бэкенде, затем ходит с токеном к API. Это делегированный доступ без передачи пароля приложению. JWT — самодостаточный токен из трёх частей (header.payload.signature), несёт claims и подписан. SA документирует flow (диаграммой), какие scope нужны, срок жизни токенов, но не реализует крипто.

SENIOR

Q84. GDPR, PCI DSS, AML/KYC: что это и как влияет на требования? Приведи конкретное требование, порождённое GDPR.

ОТВЕТ

GDPR — защита персональных данных в ЕС (согласие, право на удаление, минимизация). PCI DSS — стандарт безопасности платёжных карт (нельзя хранить CVV, шифрование карт). AML/KYC — противодействие отмыванию и 'знай своего клиента' (верификация личности, мониторинг операций). Пример GDPR-требования: 'пользователь может запросить полное удаление своих персональных данных, система удаляет/анонимизирует их за ≤ 30 дней во всех хранилищах'. Senior знает: комплаенс порождает конкретные FR и NFR.

SENIOR

Q85. Availability и отказоустойчивость: что такое Disaster Recovery, RTO/RPO и circuit breaker?

ОТВЕТ

RTO (Recovery Time Objective) — за сколько восстановимся после сбоя. RPO (Recovery Point Objective) — сколько данных допустимо потерять (глубина отката). DR — план восстановления (резервные копии, резервный ЦОД, failover). Circuit breaker — паттерн, размыкающий вызовы к упавшему сервису, чтобы не каскадировать отказ и дать ему восстановиться. Senior задаёт RTO/RPO числами, потому что от них зависит стоимость инфраструктуры.

11. Тестирование, метрики, мониторинг, DevOps (awareness)

Смежные зоны. От SA требуется понимание и корректное взаимодействие, а не исполнение.

JUNIOR

Q86. Что такое UAT и какова роль SA в нём?

ОТВЕТ

UAT (User Acceptance Testing) — приёмочное тестирование конечными пользователями/бизнесом: делает ли система то, что нужно бизнесу. SA определяет критерии приёмки, помогает спланировать сценарии и участвует в sign-off, но НЕ пишет тест-кейсы за QA. SA — про 'что считать успехом', QA — про полноту проверки.

JUNIOR

Q87. Как SA и QA делят ответственность? Опиши жизненный цикл дефекта в двух словах.

ОТВЕТ

SA определяет ожидаемое поведение (требования, AC); QA проверяет соответствие и находит дефекты. Дефект: заведён (New) → назначен и подтверждён → в работе (In Progress) → исправлен (Fixed) → перепроверен (Retest) → закрыт (Closed) или переоткрыт. SA часто подключается, когда дефект — на самом деле спор о требовании.

MIDDLE

Q88. Как убедиться, что требование тестируемо? Приведи пример нетестируемого требования и исправь его.

ОТВЕТ

Требование тестируемо, если можно однозначно проверить его выполнение. Нетестируемо: 'интерфейс должен быть удобным'. Исправление: 'новый пользователь оформляет заказ за ≤ 3 экрана и ≤ 2 минут без обращения в поддержку (проверяется на 5 пользователях)'. Каждое требование должно иметь способ проверки, иначе оно бесполезно для приёмки.

MIDDLE

Q89. Что такое три пилона наблюдаемости: логи, метрики, трейсы? Что SA специфицирует для каждого?

ОТВЕТ

Логи — дискретные события ('что случилось'). Метрики — числовые агрегаты во времени (RPS, latency, error rate). Трейсы — путь одного запроса через все сервисы ('где затормозило'). SA специфицирует: какие события логировать (особенно бизнес- и аудит-события), какие метрики и пороги алертов, нужна ли сквозная трассировка. Разница мониторинга и наблюдаемости: мониторинг — известные вопросы, observability — возможность задать новые.

MIDDLE

Q90. Что такое CI/CD? Что SA специфицирует по деплою и откату?

ОТВЕТ

CI (Continuous Integration) — частая автоматическая сборка и прогон тестов при каждом изменении. CD (Delivery/Deployment) — автоматическая доставка до окружения/прода. SA специфицирует: требования к окружениям (dev/stage/prod), критерии готовности к релизу, план и критерии отката (rollback), feature flags для постепенного раската. SA не строит пайплайн, но задаёт условия релиза.

SENIOR

Q91. Какие продуктовые и процессные метрики ты используешь, чтобы понять, удалась ли фича? Разница DAU/MAU, retention, churn, LTV.

ОТВЕТ

DAU/MAU — активные пользователи в день/месяц, их отношение показывает 'липкость'. Retention — доля вернувшихся пользователей. Churn — доля ушедших. LTV — суммарная ценность клиента за жизненный цикл. Процессные: time-to-market, lead time, cycle time, rework rate. Senior не говорит 'фича сделана', а определяет метрику успеха ЗАРАНЕЕ (как в HADI) и смотрит на неё после релиза.

SENIOR

Q92. IaaS, PaaS, SaaS: разница и как модель влияет на требования и зону ответственности?

ОТВЕТ

IaaS — арендуешь инфраструктуру (VM, сеть), сам управляешь ОС и выше (AWS EC2). PaaS — платформа, управляешь только приложением и данными (Heroku, App Engine). SaaS — готовый софт по подписке (Salesforce, Gmail). Чем выше уровень, тем меньше своя зона ответственности и гибкость, но проще эксплуатация. Senior понимает: выбор модели меняет, что вообще можно требовать и кто отвечает за доступность/безопасность.

12. Документация и инструменты

Как оформляется и версионится работа аналитика.

JUNIOR

Q93. Что такое SRS и BRD? Чем отличаются?

ОТВЕТ

BRD (Business Requirements Document) — бизнес-требования: зачем проект, цели, высокоуровневый скоуп, для бизнес-аудитории. SRS (Software Requirements Specification) — детальные функциональные и нефункциональные требования к системе, для команды разработки. BRD отвечает 'зачем и что для бизнеса', SRS — 'что именно система делает'.

MIDDLE

Q94. Как SA использует Git, если он не пишет код? Что такое ветка и pull request?

ОТВЕТ

SA версионитует спеки, ТЗ, диаграммы, схемы в Git: видна история изменений, кто и что менял, можно откатиться. Ветка (branch) — изолированная линия изменений. Pull request — предложение влить изменения с ревью и обсуждением. Это даёт требованиям тот же контроль версий и ревью, что и коду.

MIDDLE

Q95. Как связка Jira + Confluence работает в документообороте требований?

ОТВЕТ

Confluence хранит длинные документы (BRD, SRS, решения, дизайн), Jira — задачи/истории/баги для исполнения. Хорошая практика: Jira-тикет ссылается на Confluence-страницу с детальными требованиями, а страница ссылается на связанные тикеты. Так требование и работа по нему связаны и трассируемы. Confluence — стандарт на российском рынке.

Q96. Что такое ТЗ по ГОСТ 19/34 и когда оно нужно вместо гибкого SRS? В чём боль работы по ГОСТ?

ОТВЕТ

ГОСТ 19 (программная документация) и ГОСТ 34 (автоматизированные системы) задают жёсткую структуру и состав документов ТЗ, обязательны в госзаказе и многих российских энтерпрайзах. Нужно, когда требуется формальная приёмка и юридическая значимость. Боль: тяжеловесность, длинные циклы согласования, плохая совместимость с Agile, дублирование. Senior умеет работать в обоих мирах и понимает, зачем формализм там, где он есть.

13. Проверка реального опыта (не 'да/нет')

Цель: отличить реальный опыт от строчки в резюме. Проси конкретику: что сломалось, какие компромиссы, что бы сделал иначе. Задавай эти вопросы по КАЖДОЙ технологии, которую кандидат заявляет.

PROBE

Q97. Выбери одну интеграцию, которую ты реально специфицировал end-to-end. Проведи меня по ней: система-источник, протокол, формат, что пошло не так и как решили.

ЧТО СЛУШАТЬ

Слушай конкретику: реальные названия систем, обоснование протокола/формата, обработка ошибок, таймауты, ретраи, идемпотентность. Красный флаг: только счастливый путь, никаких сбоев, общие фразы. Реальный опыт всегда несёт шрамы: 'партнёр отдавал даты в своём формате', 'дубли из-за ретраев'.

PROBE

Q98. Расскажи про самое сложное требование, которое ты не смог однозначно специфицировать сразу. Почему было трудно и как ты дошёл до ясности?

ЧТО СЛУШАТЬ

Проверяем работу с неопределённостью: как копал (доп. интервью, прототип, данные), как разрешал противоречия стейкхолдеров, как фиксировал допущения. Senior показывает процесс снятия неопределённости, а не 'мне дали чёткое ТЗ'.

PROBE

Q99. Ты указал в резюме [технология X: Kafka/микросервисы/GraphQL/...]. Опиши конкретный случай, где ты с ней работал: какое решение принял и какой был trade-off?

ЧТО СЛУШАТЬ

Замени X на заявленное. Реальный пользователь объяснит контекст, альтернативы, которые отверг, и цену выбора. Поверхностный — повторит учебник без своего кейса. Дай молчание: пусть заполнит деталями. Спрашивай 'а почему не иначе?'.

PROBE

Q100. Опиши случай, когда твоё требование оказалось неверным и это дошло до разработки или прода. Что произошло и какой урок ты вынес?

ЧТО СЛУШАТЬ

Проверяем зрелость и рефлексию. Senior честно признаёт ошибку, называет корневую причину (не уточнил кейс, не поговорил с нужным стейкхолдером) и системный вывод (изменил свой процесс). Красный флаг: 'у меня не было таких случаев' или перекладывание вины целиком на других.

PROBE

Q101. Как ты решаешь конфликт, когда два ключевых стейкхолдера требуют взаимоисключающего? Приведи реальный пример и исход.

ЧТО СЛУШАТЬ

Слушай: выявление истинных интересов за позициями, использование данных/приоритизации (MoSCoW), эскалация к владельцу решения, фасилитация воркшопа. Senior не 'выбирает сторону', а делает конфликт видимым и приводит к решению с обоснованием. Важен реальный исход, а не теория.

14. AI и инструменты аналитика

Как кандидат реально усиливает себя ИИ и инструментами. Ищем конкретные workflow и критическое отношение, а не 'пользуюсь ChatGPT'.

PROBE

Q102. Как ты используешь ИИ (ChatGPT/Claude и т.п.) в ежедневной работе аналитика? Приведи конкретную задачу, которую он тебе реально ускорил.

ЧТО СЛУШАТЬ

Ищем конкретные сценарии: черновики User Story/Use Case, генерация FR/NFR, черновик SRS/TЗ, разбор большого документа, перефразирование для стейкхолдеров. Senior называет задачу, промпт-подход и как проверяет результат. Красный флаг: 'иногда спрашиваю' без конкретики.

PROBE

Q103. Как ты генерируешь диаграммы (UML/BPMN/ERD) с помощью ИИ? Какие text-to-diagram инструменты используешь?

ЧТО СЛУШАТЬ

Ищем: PlantUML или Mermaid (ИИ пишет код диаграммы из описания), затем правка. Понимание, что ИИ хорош для черновика структуры, но требует ручной проверки логики и нотации. Senior упомянет version-control-friendly подход (диаграмма как текст в Git) и где ИИ ошибается в нотации.

PROBE

Q104. Использовал ли ты ИИ для SQL и документации API? Как проверяешь, что сгенерированное верно?

ЧТО СЛУШАТЬ

Ищем: генерация/оптимизация SQL с последующей проверкой на реальной схеме и данных; генерация OpenAPI/Swagger из описания. Ключевое — критическое отношение: прогоняет запрос, проверяет план, не доверяет слепо. Senior знает про галлюцинации и не отправляет непроверенное в спеку.

PROBE

Q105. Знаешь ли разницу между чат-ботом (ChatGPT) и кодовым агентом (Claude Code, Cursor)? Использовал ли агентов, и для чего в работе аналитика?

ЧТО СЛУШАТЬ

Ищем понимание: агент читает весь проект/vault, выполняет многошаговые задачи, редактирует файлы, запускает скрипты, в отличие от чата в окне. Применения для SA: сгенерировать пачку документов из заметок, прогнать SQL, построить диаграммы, навести порядок в базе знаний. Даже если не использовал, Senior понимает разницу концептуально.

PROBE

Q106. Что такое промптинг: zero-shot, few-shot, chain-of-thought? Как ты формулируешь промпт для качественного результата?

ЧТО СЛУШАТЬ

Zero-shot — задача без примеров. Few-shot — даёшь несколько примеров нужного формата. Chain-of-thought — просишь рассуждать пошагово. Хороший промпт: роль + контекст + чёткая задача + формат вывода + примеры. Senior показывает, что относится к промпту как к постановке задачи (то, что он и так умеет как аналитик).

PROBE

Q107. Автоматизировал ли ты рутину через ИИ или no-code (например n8n): обработка встреч, транскрипция, извлечение action items, связка с Jira/Confluence/Telegram?

ЧТО СЛУШАТЬ

Ищем реальные автоматизации: транскрипция и суммаризация встреч, извлечение задач, автосоздание тикетов, боты-уведомления, кастомный GPT-агент под повторяющуюся задачу. Senior мыслит категориями 'что из моей рутины можно снять'. Даже базовый пример (ИИ-протокол встречи) ценнее теоретических рассуждений.

Q108. Какими инструментами ты пользуешься для диаграмм и документации, и как выбираешь между ними? (Draw.io, Miro, Lucidchart, Figma, Excalidraw, PlantUML/Mermaid, Confluence, Notion)

ЧТО СЛУШАТЬ

Ищем осознанный выбор под задачу: whiteboard/воркшоп (Miro, Excalidraw), строгие UML/BPMN/ERD с экспортом (Draw.io, Lucidchart), diagram-as-code в Git и в markdown (PlantUML, Mermaid), прототипы (Figma), документация (Confluence, Notion). Senior объясняет 'когда что', а не называет один инструмент на всё. Плюс интеграции (Lucidchart/Draw.io в Confluence).